

# Объектно-ориентированное программирование

## Лекция 8: Метаклассы, Декрипторы и Фабрики классов

Декрипторы ( `__get__` , `__set__` , `__delete__` ). Фабрики классов. Введение в метапрограммирование. Понятие метаклассов, `type()` . Создание пользовательских метаклассов.



```
1 class Product:
2     def __init__(self, name, price, quantity):
3         self.name = name
4         self.price = price
5         self.quantity = quantity
```

```
1 class Product:
2     def __init__(self, name, price, quantity):
3         self.name = name
4         self.price = price
5         self.quantity = quantity
6
7     @property
8     def price(self):
9         return self._price
10
11    @price.setter
12    def price(self, value):
13        if not isinstance(value, (int, float)):
14            raise TypeError
15        if value < 0:
16            raise ValueError
17        self._price = value
```



```
1 class Product:
2     def __init__(self, name, price, quantity):
3         self.name = name
4         self.price = price
5         self.quantity = quantity
6
7     ...
8
9     @property
10    def quantity(self):
11        return self._quantity
12
13    @quantity.setter
14    def quantity(self, value):
15        if not isinstance(value, int):
16            raise TypeError
17        self._quantity = value
```

**Дескриптор** — это класс, который реализует хотя бы один из методов протокола дескриптора: `__get__`, `__set__` или `__delete__`. Экземпляр этого класса назначается как атрибут **другого** класса.

*Дескриптор — это «менеджер» конкретной переменной. Сам объект не хранит данные и не проверяет их, он делегирует эту работу менеджеру (Дескриптору).*

| Метод | Сигнатура | Когда вызывается | Описание аргументов |
|-------|-----------|------------------|---------------------|
|       |           |                  |                     |
|       |           |                  |                     |
|       |           |                  |                     |
|       |           |                  |                     |
|       |           |                  |                     |



# Протокол дескриптора (Методы)

| Метод      | Сигнатура                                   | Когда вызывается                                     | Описание аргументов                                                                                                                                                |
|------------|---------------------------------------------|------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Get</b> | <code>__get__(self, instance, owner)</code> | При чтении атрибута:<br><code>print(obj.attr)</code> | <code>self</code> — сам дескриптор.<br><code>instance</code> — объект, у которого читают атрибут (Студент).<br><code>owner</code> — класс объекта (class Student). |
|            |                                             |                                                      |                                                                                                                                                                    |
|            |                                             |                                                      |                                                                                                                                                                    |
|            |                                             |                                                      |                                                                                                                                                                    |
|            |                                             |                                                      |                                                                                                                                                                    |
|            |                                             |                                                      |                                                                                                                                                                    |



# Протокол дескриптора (Методы)

| Метод      | Сигнатура                                   | Когда вызывается                                     | Описание аргументов                                                                                                                                                |
|------------|---------------------------------------------|------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Get</b> | <code>__get__(self, instance, owner)</code> | При чтении атрибута:<br><code>print(obj.attr)</code> | <code>self</code> — сам дескриптор.<br><code>instance</code> — объект, у которого читают атрибут (Студент).<br><code>owner</code> — класс объекта (class Student). |
| <b>Set</b> | <code>__set__(self, instance, value)</code> | При записи:<br><code>obj.attr = 10</code>            | <code>value</code> — новое значение, которое пытаются присвоить.                                                                                                   |





## Протокол дескриптора (Методы)

| Метод         | Сигнатура                                   | Когда вызывается                                     | Описание аргументов                                                                                                                                                               |
|---------------|---------------------------------------------|------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Get</b>    | <code>__get__(self, instance, owner)</code> | При чтении атрибута:<br><code>print(obj.attr)</code> | <code>self</code> — сам дескриптор.<br><code>instance</code> — объект, у которого читают атрибут (Студент).<br><code>owner</code> — класс объекта ( <code>class Student</code> ). |
| <b>Set</b>    | <code>__set__(self, instance, value)</code> | При записи:<br><code>obj.attr = 10</code>            | <code>value</code> — новое значение, которое пытаются присвоить.                                                                                                                  |
| <b>Delete</b> | <code>__delete__(self, instance)</code>     | При удалении:<br><code>del obj.attr</code>           | <code>instance</code> — объект, у которого удаляют атрибут.                                                                                                                       |



# Протокол дескриптора (Методы)

| Метод           | Сигнатура                                    | Когда вызывается                                     | Описание аргументов                                                                                                                                                |
|-----------------|----------------------------------------------|------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Get</b>      | <code>__get__(self, instance, owner)</code>  | При чтении атрибута:<br><code>print(obj.attr)</code> | <code>self</code> — сам дескриптор.<br><code>instance</code> — объект, у которого читают атрибут (Студент).<br><code>owner</code> — класс объекта (class Student). |
| <b>Set</b>      | <code>__set__(self, instance, value)</code>  | При записи:<br><code>obj.attr = 10</code>            | <code>value</code> — новое значение, которое пытаются присвоить.                                                                                                   |
| <b>Delete</b>   | <code>__delete__(self, instance)</code>      | При удалении:<br><code>del obj.attr</code>           | <code>instance</code> — объект, у которого удаляют атрибут.                                                                                                        |
| <b>Set Name</b> | <code>__set_name__(self, owner, name)</code> | При создании класса (Python 3.6+)                    | Автоматически сообщает дескриптору имя переменной, к которой он привязан.                                                                                          |



Университет  
**Сириус**  
Колледж

# Где хранить данные?



## Где хранить данные?

- **Ошибка:** Хранить значение в `self.value` внутри дескриптора. Значение станет общим для **всех** экземпляров класса (так как дескриптор создается один раз на уровне класса).



## Где хранить данные?

- **Ошибка:** Хранить значение в `self.value` внутри дескриптора. Значение станет общим для **всех** экземпляров класса (так как дескриптор создается один раз на уровне класса).
- **Правило:** Дескриптор должен хранить данные внутри `instance` (экземпляра объекта, который его использует), обычно в словаре `instance.__dict__`.



```
1 class Integer:
2     def __set_name__(self, owner, name):
3         self.name = "_" + name
4
5     def __get__(self, instance, owner):
6         if instance is None:
7             return self
8         return getattr(instance, self.name, 0)
9
10    def __set__(self, instance, value):
11        print(f"Установка значения для {self.name}")
12        if not isinstance(value, int):
13            raise TypeError("Значение должно быть целым числом")
14        if value < 0:
15            raise ValueError("Значение не может быть отрицательным")
```



```
1 class Integer:
2     def __set_name__(self, owner, name):
3         self.name = "_" + name
4
5     def __get__(self, instance, owner):
6         if instance is None:
7             return self
8         return getattr(instance, self.name, 0)
9
10    def __set__(self, instance, value):
11        print(f"Установка значения для {self.name}")
12        if not isinstance(value, int):
13            raise TypeError("Значение должно быть целым числом")
14        if value < 0:
15            raise ValueError("Значение не может быть отрицательным")
```



```
1 class Integer:
2     def __set_name__(self, owner, name):
3         self.name = "_" + name
4
5     def __get__(self, instance, owner):
6         if instance is None:
7             return self
8         return getattr(instance, self.name, 0)
9
10    def __set__(self, instance, value):
11        print(f"Установка значения для {self.name}")
12        if not isinstance(value, int):
13            raise TypeError("Значение должно быть целым числом")
14        if value < 0:
15            raise ValueError("Значение не может быть отрицательным")
```





```
1 class Integer:
2     def __set_name__(self, owner, name):
3         self.name = "_" + name
4
5     def __get__(self, instance, owner):
6         if instance is None:
7             return self
8         return getattr(instance, self.name, 0)
9
10    def __set__(self, instance, value):
11        print(f"Установка значения для {self.name}")
12        if not isinstance(value, int):
13            raise TypeError("Значение должно быть целым числом")
14        if value < 0:
15            raise ValueError("Значение не может быть отрицательным")
```



```
1 class Integer:
2     def __set_name__(self, owner, name):
3         self.name = "_" + name
4
5     def __get__(self, instance, owner):
6         if instance is None:
7             return self
8         return getattr(instance, self.name, 0)
9
10    def __set__(self, instance, value):
11        print(f"Установка значения для {self.name}")
12        if not isinstance(value, int):
13            raise TypeError("Значение должно быть целым числом")
14        if value < 0:
15            raise ValueError("Значение не может быть отрицательным")
```



```
1 class Integer:
2     def __set_name__(self, owner, name):
3         self.name = "_" + name
4
5     def __get__(self, instance, owner):
6         if instance is None:
7             return self
8         return getattr(instance, self.name, 0)
9
10    def __set__(self, instance, value):
11        print(f"Установка значения для {self.name}")
12        if not isinstance(value, int):
13            raise TypeError("Значение должно быть целым числом")
14        if value < 0:
15            raise ValueError("Значение не может быть отрицательным")
```



```
7         return self
8     return getattr(instance, self.name, 0)
9
10    def __set__(self, instance, value):
11        print(f"Установка значения для {self.name}")
12        if not isinstance(value, int):
13            raise TypeError("Значение должно быть целым числом")
14        if value < 0:
15            raise ValueError("Значение не может быть отрицательным")
16
17        setattr(instance, self.name, value)
18
19    class Point:
20        x = Integer()
21        y = Integer()
```



```
7         return self
8     return getattr(instance, self.name, 0)
9
10    def __set__(self, instance, value):
11        print(f"Установка значения для {self.name}")
12        if not isinstance(value, int):
13            raise TypeError("Значение должно быть целым числом")
14        if value < 0:
15            raise ValueError("Значение не может быть отрицательным")
16
17        setattr(instance, self.name, value)
18
19    class Point:
20        x = Integer()
21        y = Integer()
```



```
10     def __set__(self, instance, value):
11         print(f"Установка значения для {self.name}")
12         if not isinstance(value, int):
13             raise TypeError("Значение должно быть целым числом")
14         if value < 0:
15             raise ValueError("Значение не может быть отрицательным")
16
17         setattr(instance, self.name, value)
18
19 class Point:
20     x = Integer()
21     y = Integer()
22
23     def __init__(self, x, y):
24         self.x = x
```



```
15         raise ValueError("Значение не может быть отрицательным")
16
17         setattr(instance, self.name, value)
18
19 class Point:
20     x = Integer()
21     y = Integer()
22
23     def __init__(self, x, y):
24         self.x = x
25         self.y = y
26
27 p1 = Point(10, 20)
28 p1.x = 100 # Вызовет __set__ у дескриптора Integer
29 p1.y = "строка" # Вызовет TypeError
```



```
14         if value < 0:
15             raise ValueError("Значение не может быть отрицательным")
16
17         setattr(instance, self.name, value)
18
19 class Point:
20     x = Integer()
21     y = Integer()
22
23     def __init__(self, x, y):
24         self.x = x
25         self.y = y
26
27 p1 = Point(10, 20)
28 p1.x = 100 # Вызовет set у дескриптора Integer
```





```
15         raise ValueError( 'значение не может быть отрицательным' )
16
17         setattr(instance, self.name, value)
18
19 class Point:
20     x = Integer()
21     y = Integer()
22
23     def __init__(self, x, y):
24         self.x = x
25         self.y = y
26
27 p1 = Point(10, 20)
28 p1.x = 100 # Вызовет __set__ у дескриптора Integer
29 p1.y = "строка" # Вызовет TypeError
```



```
15         raise ValueError( значение не может быть отрицательным )
16
17         setattr(instance, self.name, value)
18
19 class Point:
20     x = Integer()
21     y = Integer()
22
23     def __init__(self, x, y):
24         self.x = x
25         self.y = y
26
27 p1 = Point(10, 20)
28 p1.x = 100 # Вызовет __set__ у дескриптора Integer
29 p1.y = "строка" # Вызовет TypeError
```



Университет  
**Сириус**  
Колледж

# Виды дескрипторов (Data vs Non-Data)

1. **Дескрипторы данных (Data Descriptors):** Реализуют `__set__` или `__delete__`.

1. **Дескрипторы данных (Data Descriptors):** Реализуют `__set__` или `__delete__`.
  - Имеют приоритет над словарем объекта `__dict__`. Даже если в объекте есть атрибут с таким именем, Python все равно вызовет дескриптор.

## Виды дескрипторов (Data vs Non-Data)

- 1. Дескрипторы данных (Data Descriptors):** Реализуют `__set__` или `__delete__`.
  - Имеют приоритет над словарем объекта `__dict__`. Даже если в объекте есть атрибут с таким именем, Python все равно вызовет дескриптор.
- 2. Дескрипторы без данных (Non-Data Descriptors):** Реализуют только `__get__`.

# Виды дескрипторов (Data vs Non-Data)

1. **Дескрипторы данных (Data Descriptors):** Реализуют `__set__` или `__delete__`.
  - Имеют приоритет над словарем объекта `__dict__`. Даже если в объекте есть атрибут с таким именем, Python все равно вызовет дескриптор.
2. **Дескрипторы без данных (Non-Data Descriptors):** Реализуют только `__get__`.
  - Имеют приоритет **ниже**, чем `__dict__`.

# Виды дескрипторов (Data vs Non-Data)

1. **Дескрипторы данных (Data Descriptors):** Реализуют `__set__` или `__delete__`.
  - Имеют приоритет над словарем объекта `__dict__`. Даже если в объекте есть атрибут с таким именем, Python все равно вызовет дескриптор.
2. **Дескрипторы без данных (Non-Data Descriptors):** Реализуют только `__get__`.
  - Имеют приоритет **ниже**, чем `__dict__`.
  - *Пример:* Обычные методы в Python — это дескрипторы без данных (они превращают функцию в связанный метод при доступе).



1. `@property`, `@classmethod`, `@staticmethod` — это всё дескрипторы "под капотом".
2. **ORM (Django, SQLAlchemy):** Когда в Django пишут `name = models.CharField( ... )`, `CharField` — это мощный дескриптор, который отвечает за валидацию, связь с базой данных и отслеживание изменений.



```
1 class Order:
2     @property
3     def price(self): ...
4     @price.setter
5     def price(self, val): # проверка > 0
6
7     @property
8     def quantity(self): ...
9     @quantity.setter
10    def quantity(self, val): # та же проверка > 0!
11
12    # И так много раз...
```

```
1 class PositiveNumber:
2     # Логика проверки 1 раз
3     ...
4
5 class Order:
6     price = PositiveNumber()
7     quantity = PositiveNumber()
8     discount = PositiveNumber()
```

**Фабрика классов** — это функция или метод, которые создают и возвращают **новый класс**.

В Python класс — это тоже объект (экземпляр метакласса `type`). Значит, мы можем:

1. Создать класс внутри функции.
2. Вернуть класс как результат работы функции.
3. Присвоить класс переменной.



## Реализация №1: Функция-обертка

Самый простой способ создать фабрику классов — определить класс внутри функции.

```
1  def create_robot_class(robot_type):
2      """Фабрика, создающая классы роботов с разным приветствием."""
3
4      class Robot:
5          def __init__(self, name):
6              self.name = name
7
8          def say_hello(self):
9              if robot_type == "war":
10                 return f"{self.name}: Готов уничтожать!"
11                 elif robot_type == "service":
12                     return f"{self.name}: Чем могу помочь?"
13                 else:
```



## Реализация №1: Функция-обертка

Самый простой способ создать фабрику классов — определить класс внутри функции.

```
1  def create_robot_class(robot_type):
2      """Фабрика, создающая классы роботов с разным приветствием."""
3
4      class Robot:
5          def __init__(self, name):
6              self.name = name
7
8          def say_hello(self):
9              if robot_type == "war":
10                 return f"{self.name}: Готов уничтожать!"
11             elif robot_type == "service":
12                 return f"{self.name}: Чем могу помочь?"
13             else:
```



## Реализация №1: Функция-обертка

Самый простой способ создать фабрику классов — определить класс внутри функции.

```
3
4     class Robot:
5         def __init__(self, name):
6             self.name = name
7
8         def say_hello(self):
9             if robot_type == "war":
10                 return f"{self.name}: Готов уничтожать!"
11             elif robot_type == "service":
12                 return f"{self.name}: Чем могу помочь?"
13             else:
14                 return f"{self.name}: Бип-буп."
15
```



## Реализация №1: Функция-обертка

Самый простой способ создать фабрику классов — определить класс внутри функции.

```
3
4     class Robot:
5         def __init__(self, name):
6             self.name = name
7
8         def say_hello(self):
9             if robot_type == "war":
10                 return f"{self.name}: Готов уничтожать!"
11             elif robot_type == "service":
12                 return f"{self.name}: Чем могу помочь?"
13             else:
14                 return f"{self.name}: Бип-буп."
15
```





## Реализация №1: Функция-обертка

Самый простой способ создать фабрику классов — определить класс внутри функции.

```
11         elif robot_type == "service":
12             return f"{self.name}: Чем могу помочь?"
13         else:
14             return f"{self.name}: Бип-буп."
15
16     # Возвращаем сам КЛАСС (не экземпляр!)
17     return Robot
18
19 WarRobot = create_robot_class("war")           # WarRobot теперь — это класс
20 ServiceRobot = create_robot_class("service")  # ServiceRobot — другой класс
21
22 terminator = WarRobot("T-800")
23 wall_e = ServiceRobot("Wall-E")
```



## Реализация №1: Функция-обертка

Самый простой способ создать фабрику классов — определить класс внутри функции.

```
14         return f"{self.name}: Бип-буп."
15
16     # Возвращаем сам КЛАСС (не экземпляр!)
17     return Robot
18
19 WarRobot = create_robot_class("war")           # WarRobot теперь — это класс
20 ServiceRobot = create_robot_class("service")   # ServiceRobot — другой класс
21
22 terminator = WarRobot("T-800")
23 wall_e = ServiceRobot("Wall-E")
24
25 print(terminator.say_hello()) # T-800: Готов уничтожать!
26 print(wall_e.say_hello())    # Wall-E: Чем могу помочь?
```



## Реализация №1: Функция-обертка

Самый простой способ создать фабрику классов — определить класс внутри функции.

```
14         return f"{self.name}: Бип-буп."
15
16     # Возвращаем сам КЛАСС (не экземпляр!)
17     return Robot
18
19 WarRobot = create_robot_class("war")           # WarRobot теперь — это класс
20 ServiceRobot = create_robot_class("service") # ServiceRobot — другой класс
21
22 terminator = WarRobot("T-800")
23 wall_e = ServiceRobot("Wall-E")
24
25 print(terminator.say_hello()) # T-800: Готов уничтожать!
26 print(wall_e.say_hello())    # Wall-E: Чем могу помочь?
```

Функция `type` имеет два применения:

1. `type(obj)` — возвращает тип объекта.
2. `type(name, bases, dict)` — **создает новый класс**.
  - `name ( str )`: Имя будущего класса (то, что будет в `__name__` ).
  - `bases ( tuple )`: Кортеж родительских классов (наследование).
  - `dict ( dict )`: Пространство имен класса (атрибуты и методы).

## Реализация №2: Использование type()

```
1 def say_hello_func(self):
2     return f"Привет, я {self.name}"
3
4 User = type(
5     'User',          # Имя класса
6     (),              # Родители (пустой кортеж)
7     {                 # Атрибуты и методы
8         'category': 'person',
9         '__init__': lambda self, name: setattr(self, 'name', name),
10        'hello': say_hello_func
11    }
12 )
13
```

## Реализация №2: Использование type()

```
2     return f"Привет, я {self.name}"
3
4 User = type(
5     'User',                # Имя класса
6     (),                    # Родители (пустой кортеж)
7     {                      # Атрибуты и методы
8         'category': 'person',
9         '__init__': lambda self, name: setattr(self, 'name', name),
10        'hello': say_hello_func
11    }
12 )
13
14 u = User("Alice")
```



## Реализация №2: Использование type()

```
1 def say_hello_func(self):
2     return f"Привет, я {self.name}"
3
4 User = type(
5     'User',          # Имя класса
6     (),              # Родители (пустой кортеж)
7     {                 # Атрибуты и методы
8         'category': 'person',
9         '__init__': lambda self, name: setattr(self, 'name', name),
10        'hello': say_hello_func
11    }
12 )
13
```



## Реализация №2: Использование type()

```
1 def say_hello_func(self):
2     return f"Привет, я {self.name}"
3
4 User = type(
5     'User',                # Имя класса
6     (),                    # Родители (пустой кортеж)
7     {                       # Атрибуты и методы
8         'category': 'person',
9         '__init__': lambda self, name: setattr(self, 'name', name),
10        'hello': say_hello_func
11    }
12 )
13
```





## Реализация №2: Использование type()

```
3
4 User = type(
5     'User',                      # Имя класса
6     (),                          # Родители (пустой кортеж)
7     {                             # Атрибуты и методы
8         'category': 'person',
9         '__init__': lambda self, name: setattr(self, 'name', name),
10        'hello': say_hello_func
11    }
12 )
13
14 u = User("Alice")
15 print(u.hello())                # Привет, я Alice
```



## Реализация №2: Использование type()

```
2     return f"Привет, я {self.name}"
3
4 User = type(
5     'User',                # Имя класса
6     (),                    # Родители (пустой кортеж)
7     {                       # Атрибуты и методы
8         'category': 'person',
9         '__init__': lambda self, name: setattr(self, 'name', name),
10        'hello': say_hello_func
11    }
12 )
13
14 u = User("Alice")
```

```
3
4 User = type(
5     'User',                      # Имя класса
6     (),                          # Родители (пустой кортеж)
7     {                            # Атрибуты и методы
8         'category': 'person',
9         '__init__': lambda self, name: setattr(self, 'name', name),
10        'hello': say_hello_func
11    }
12 )
13
14 u = User("Alice")
15 print(u.hello())                # Привет, я Alice
```



## Реализация №2: Использование type()

```
1 def say_hello_func(self):
2     return f"Привет, я {self.name}"
3
4 User = type(
5     'User',          # Имя класса
6     (),              # Родители (пустой кортеж)
7     {                # Атрибуты и методы
8         'category': 'person',
9         '__init__': lambda self, name: setattr(self, 'name', name),
10        'hello': say_hello_func
11    }
12 )
13
```



## Реализация №2: Использование type()

```
4  class User:
5      'User',                # Имя класса
6      (),                    # Родители (пустой кортеж)
7      {                       # Атрибуты и методы
8          'category': 'person',
9          '__init__': lambda self, name: setattr(self, 'name', name),
10         'hello': say_hello_func
11     }
12 )
13
14 u = User("Alice")
15 print(u.hello())           # Привет, я Alice
16 print(type(u))             # <class '__main__.User'>
```

## Реализация №2: Использование type()

```
4  class User:
5      'User',          # Имя класса
6      (),              # Родители (пустой кортеж)
7      {                # Атрибуты и методы
8          'category': 'person',
9          '__init__': lambda self, name: setattr(self, 'name', name),
10         'hello': say_hello_func
11     }
12 )
13
14 u = User("Alice")
15 print(u.hello())      # Привет, я Alice
16 print(type(u))        # <class '__main__.User'>
```

**Пример из жизни:** `collections.namedtuple` — это классическая фабрика классов в стандартной библиотеке.



Университет  
**Сириус**  
Колледж

# Метапрограммирование

**Метапрограммирование** — это написание программ, которые управляют другими программами (или самими собой). В контексте Python это означает **перехват и изменение процесса создания классов**.

**Метапрограммирование** — это написание программ, которые управляют другими программами (или самими собой). В контексте Python это означает **перехват и изменение процесса создания классов**.

**Аналогия "Печенье и Формочка":**



**Метапрограммирование** — это написание программ, которые управляют другими программами (или самими собой). В контексте Python это означает **перехват и изменение процесса создания классов**.

**Аналогия "Печенье и Формочка":**

1. **Объект** (instance) — это **Печенье**.
2. **Класс** (class) — это **Формочка** для печенья (инструкция, как делать объекты).
3. **Метакласс** (metaclass) — это **Завод по производству формочек**.

**Метапрограммирование** — это написание программ, которые управляют другими программами (или самими собой). В контексте Python это означает **перехват и изменение процесса создания классов**.

**Аналогия "Печенье и Формочка":**

1. **Объект** (instance) — это **Печенье**.
2. **Класс** (class) — это **Формочка** для печенья (инструкция, как делать объекты).
3. **Метакласс** (metaclass) — это **Завод по производству формочек**.

*Метакласс создает классы. Классы создают объекты.*

`type` — это встроенный метакласс по умолчанию. Все классы в Python являются экземплярами `type`.

```
1 class A: pass
2
3 print(type(A())) # <class '__main__.A'> (Объект создан классом A)
4 print(type(A))   # <class 'type'>      (Класс A создан метаклассом type)
```

Чтобы изменить поведение создания класса, нужно наследоваться от `type`.

## Основные методы метакласса:

1. `__new__(mcs, name, bases, attrs):`
  - Вызывается **до** создания класса.
  - Именно здесь мы можем изменить атрибуты, добавить методы или проверить имя класса.
2. `__init__(cls, name, bases, attrs):`
  - Вызывается **после** создания класса (для инициализации).

```
1 class CamelCaseMeta(type):
2     def __new__(mcs, name, bases, attrs):
3         # mcs - это сам метакласс
4         # name - имя создаваемого класса (например, "my_class")
5
6         print(f"Метакласс: Проверяю имя класса '{name}'...")
7
8         if not name[0].isupper():
9             raise TypeError(f"Класс {name} должен начинаться с большой буквы!")
10
11        return super().__new__(mcs, name, bases, attrs)
12
13 class GoodClass(metaclass=CamelCaseMeta):
14     pass
15 # Вывод: Метакласс: Проверяю имя класса 'GoodClass' (Успех)
```

```
1 class CamelCaseMeta(type):
2     def __new__(mcs, name, bases, attrs):
3         # mcs - это сам метакласс
4         # name - имя создаваемого класса (например, "my_class")
5
6         print(f"Метакласс: Проверяю имя класса '{name}'...")
7
8         if not name[0].isupper():
9             raise TypeError(f"Класс {name} должен начинаться с большой буквы!")
10
11         return super().__new__(mcs, name, bases, attrs)
12
13 class GoodClass(metaclass=CamelCaseMeta):
14     pass
15 # Вывод: Метакласс: Проверяю имя класса 'GoodClass' (Успех)
```

```
1 class CamelCaseMeta(type):
2     def __new__(mcs, name, bases, attrs):
3         # mcs - это сам метакласс
4         # name - имя создаваемого класса (например, "my_class")
5
6         print(f"Метакласс: Проверяю имя класса '{name}'...")
7
8         if not name[0].isupper():
9             raise TypeError(f"Класс {name} должен начинаться с большой буквы!")
10
11        return super().__new__(mcs, name, bases, attrs)
12
13 class GoodClass(metaclass=CamelCaseMeta):
14     pass
15 # Вывод: Метакласс: Проверяю имя класса 'GoodClass' (Успех)
```

```
1 class CamelCaseMeta(type):
2     def __new__(mcs, name, bases, attrs):
3         # mcs - это сам метакласс
4         # name - имя создаваемого класса (например, "my_class")
5
6         print(f"Метакласс: Проверяю имя класса '{name}'...")
7
8         if not name[0].isupper():
9             raise TypeError(f"Класс {name} должен начинаться с большой буквы!")
10
11         return super().__new__(mcs, name, bases, attrs)
12
13 class GoodClass(metaclass=CamelCaseMeta):
14     pass
15 # Вывод: Метакласс: Проверяю имя класса 'GoodClass' (Успех)
```





```
2     def __new__(mcs, name, bases, attrs):
3         # mcs - это сам метакласс
4         # name - имя создаваемого класса (например, "my_class")
5
6         print(f"Метакласс: Проверяю имя класса '{name}'...")
7
8         if not name[0].isupper():
9             raise TypeError(f"Класс {name} должен начинаться с большой буквы!")
10
11         return super().__new__(mcs, name, bases, attrs)
12
13 class GoodClass(metaclass=CamelCaseMeta):
14     pass
15 # Вывод: Метакласс: Проверяю имя класса 'GoodClass'... (Успех)
16
```

```
4         # name - имя создаваемого класса (например, "my_class")
5
6         print(f"Метакласс: Проверяю имя класса '{name}'...")
7
8         if not name[0].isupper():
9             raise TypeError(f"Класс {name} должен начинаться с большой буквы!")
10
11        return super().__new__(mcs, name, bases, attrs)
12
13 class GoodClass(metaclass=CamelCaseMeta):
14     pass
15 # Вывод: Метакласс: Проверяю имя класса 'GoodClass'... (Успех)
16
17 class bad_class(metaclass=CamelCaseMeta):
18     pass
```

```
5
6     print(f"Метакласс: Проверяю имя класса '{name}'...")
7
8     if not name[0].isupper():
9         raise TypeError(f"Класс {name} должен начинаться с большой буквы!")
10
11     return super().__new__(mcs, name, bases, attrs)
12
13 class GoodClass(metaclass=CamelCaseMeta):
14     pass
15 # Вывод: Метакласс: Проверяю имя класса 'GoodClass'... (Успех)
16
17 class bad_class(metaclass=CamelCaseMeta):
18     pass
19 # Ошибка: TypeError: Класс bad_class должен начинаться с большой буквы!
```

```
5
6     print(f"Метакласс: Проверяю имя класса '{name}'...")
7
8     if not name[0].isupper():
9         raise TypeError(f"Класс {name} должен начинаться с большой буквы!")
10
11     return super().__new__(mcs, name, bases, attrs)
12
13 class GoodClass(metaclass=CamelCaseMeta):
14     pass
15 # Вывод: Метакласс: Проверяю имя класса 'GoodClass'... (Успех)
16
17 class bad_class(metaclass=CamelCaseMeta):
18     pass
19 # Ошибка: TypeError: Класс bad_class должен начинаться с большой буквы!
```

# Практический пример

## Автоматическая регистрация (Registry)

```
1 class PluginMeta(type):
2     plugins = []
3
4     def __init__(cls, name, bases, attrs):
5         super().__init__(name, bases, attrs)
6
7         if name != "Plugin":
8             print(f"Регистрация плагина: {name}")
9             PluginMeta.plugins.append(cls)
10
11 class Plugin(metaclass=PluginMeta):
12     """Базовый класс для всех плагинов"""
13     def action(self): pass
14
15 class AudioPlugin(Plugin):
```

# Практический пример

## Автоматическая регистрация (Registry)

```
1 class PluginMeta(type):
2     plugins = []
3
4     def __init__(cls, name, bases, attrs):
5         super().__init__(name, bases, attrs)
6
7         if name != "Plugin":
8             print(f"Регистрация плагина: {name}")
9             PluginMeta.plugins.append(cls)
10
11 class Plugin(metaclass=PluginMeta):
12     """Базовый класс для всех плагинов"""
13     def action(self): pass
14
15 class AudioPlugin(Plugin):
```

# Практический пример

## Автоматическая регистрация (Registry)

```
1 class PluginMeta(type):
2     plugins = []
3
4     def __init__(cls, name, bases, attrs):
5         super().__init__(name, bases, attrs)
6
7         if name != "Plugin":
8             print(f"Регистрация плагина: {name}")
9             PluginMeta.plugins.append(cls)
10
11 class Plugin(metaclass=PluginMeta):
12     """Базовый класс для всех плагинов"""
13     def action(self): pass
14
15 class AudioPlugin(Plugin):
```

# Практический пример

## Автоматическая регистрация (Registry)

```
1 class PluginMeta(type):
2     plugins = []
3
4     def __init__(cls, name, bases, attrs):
5         super().__init__(name, bases, attrs)
6
7         if name != "Plugin":
8             print(f"Регистрация плагина: {name}")
9             PluginMeta.plugins.append(cls)
10
11 class Plugin(metaclass=PluginMeta):
12     """Базовый класс для всех плагинов"""
13     def action(self): pass
14
15 class AudioPlugin(Plugin):
```



# Практический пример

## Автоматическая регистрация (Registry)

```
1 class PluginMeta(type):
2     plugins = []
3
4     def __init__(cls, name, bases, attrs):
5         super().__init__(name, bases, attrs)
6
7         if name != "Plugin":
8             print(f"Регистрация плагина: {name}")
9             PluginMeta.plugins.append(cls)
10
11 class Plugin(metaclass=PluginMeta):
12     """Базовый класс для всех плагинов"""
13     def action(self): pass
14
15 class AudioPlugin(Plugin):
```

# Практический пример

## Автоматическая регистрация (Registry)

```
1 class PluginMeta(type):
2     plugins = []
3
4     def __init__(cls, name, bases, attrs):
5         super().__init__(name, bases, attrs)
6
7         if name != "Plugin":
8             print(f"Регистрация плагина: {name}")
9             PluginMeta.plugins.append(cls)
10
11 class Plugin(metaclass=PluginMeta):
12     """Базовый класс для всех плагинов"""
13     def action(self): pass
14
15 class AudioPlugin(Plugin):
```

# Практический пример

## Автоматическая регистрация (Registry)

```
1 class PluginMeta(type):
2     plugins = []
3
4     def __init__(cls, name, bases, attrs):
5         super().__init__(name, bases, attrs)
6
7         if name != "Plugin":
8             print(f"Регистрация плагина: {name}")
9             PluginMeta.plugins.append(cls)
10
11 class Plugin(metaclass=PluginMeta):
12     """Базовый класс для всех плагинов"""
13     def action(self): pass
14
15 class AudioPlugin(Plugin):
```

# Практический пример

## Автоматическая регистрация (Registry)

```
1 class PluginMeta(type):
2     plugins = []
3
4     def __init__(cls, name, bases, attrs):
5         super().__init__(name, bases, attrs)
6
7         if name != "Plugin":
8             print(f"Регистрация плагина: {name}")
9             PluginMeta.plugins.append(cls)
10
11 class Plugin(metaclass=PluginMeta):
12     """Базовый класс для всех плагинов"""
13     def action(self): pass
14
15 class AudioPlugin(Plugin):
```

# Практический пример

## Автоматическая регистрация (Registry)

```
2     plugins = []
3
4     def __init__(cls, name, bases, attrs):
5         super().__init__(name, bases, attrs)
6
7         if name != "Plugin":
8             print(f"Регистрация плагина: {name}")
9             PluginMeta.plugins.append(cls)
10
11 class Plugin(metaclass=PluginMeta):
12     """Базовый класс для всех плагинов"""
13     def action(self): pass
14
15 class AudioPlugin(Plugin):
16     def action(self): print("Playing Audio")
```

# Практический пример

## Автоматическая регистрация (Registry)

```
5         super().__init__(name, bases, attrs)
6
7         if name != "Plugin":
8             print(f"Регистрация плагина: {name}")
9             PluginMeta.plugins.append(cls)
10
11 class Plugin(metaclass=PluginMeta):
12     """Базовый класс для всех плагинов"""
13     def action(self): pass
14
15 class AudioPlugin(Plugin):
16     def action(self): print("Playing Audio")
17
18 class VideoPlugin(Plugin):
19     def action(self): print("Playing Video")
```

# Практический пример

## Автоматическая регистрация (Registry)

```
9         PluginMeta.plugins.append(cls)
10
11 class Plugin(metaclass=PluginMeta):
12     """Базовый класс для всех плагинов"""
13     def action(self): pass
14
15 class AudioPlugin(Plugin):
16     def action(self): print("Playing Audio")
17
18 class VideoPlugin(Plugin):
19     def action(self): print("Playing Video")
20
21
22 print("\nСписок плагинов:", PluginMeta.plugins)
23 # [AudioPlugin, VideoPlugin]
```

# Практический пример

## Автоматическая регистрация (Registry)

```
9         PluginMeta.plugins.append(cls)
10
11 class Plugin(metaclass=PluginMeta):
12     """Базовый класс для всех плагинов"""
13     def action(self): pass
14
15 class AudioPlugin(Plugin):
16     def action(self): print("Playing Audio")
17
18 class VideoPlugin(Plugin):
19     def action(self): print("Playing Video")
20
21
22 print("\nСписок плагинов:", PluginMeta.plugins)
23 # [AudioPlugin, VideoPlugin]
```



## Практический пример Автоматическая регистрация (Registry)

```
9         PluginMeta.plugins.append(cls)
10
11 class Plugin(metaclass=PluginMeta):
12     """Базовый класс для всех плагинов"""
13     def action(self): pass
14
15 class AudioPlugin(Plugin):
16     def action(self): print("Playing Audio")
17
18 class VideoPlugin(Plugin):
19     def action(self): print("Playing Video")
20
21
22 print("\nСписок плагинов:", PluginMeta.plugins)
23 # [AudioPlugin, VideoPlugin]
```



*"Метаклассы — это магия на 99%. Если вы думаете, нужны ли они вам, то они вам не нужны. Те, кому они действительно нужны, точно знают зачем, и им не нужно объяснять почему." Тим Питерс (автор *Zen of Python*)*

*"Метаклассы — это магия на 99%. Если вы думаете, нужны ли они вам, то они вам не нужны. Те, кому они действительно нужны, точно знают зачем, и им не нужно объяснять почему." Тим Питерс (автор *Zen of Python*)*

**Когда использовать метапрограммирование?**

*"Метаклассы — это магия на 99%. Если вы думаете, нужны ли они вам, то они вам не нужны. Те, кому они действительно нужны, точно знают зачем, и им не нужно объяснять почему." Тим Питерс (автор Zen of Python)*

## Когда использовать метапрограммирование?

1. При написании фреймворков и библиотек (ORM, API).
2. Когда нужно гарантировать строгое соблюдение правил в большом количестве классов.
3. Для автоматической регистрации и уменьшения boilerplate-кода.

*"Метаклассы — это магия на 99%. Если вы думаете, нужны ли они вам, то они вам не нужны. Те, кому они действительно нужны, точно знают зачем, и им не нужно объяснять почему." Тим Питерс (автор Zen of Python)*

## Когда использовать метапрограммирование?

1. При написании фреймворков и библиотек (ORM, API).
2. Когда нужно гарантировать строгое соблюдение правил в большом количестве классов.
3. Для автоматической регистрации и уменьшения boilerplate-кода.

В обычном прикладном коде лучше использовать более простые инструменты (декораторы, наследование).